benchmarks

September 13, 2025

[1]:
```
from pynamicalsys import DiscreteDynamicalSystem as dds
from pynamicalsys import ContinuousDynamicalSystem as cds
from pynamicalsys import HamiltonianSystem as HS
```

[2]:
```
import numpy as np
from numba import njit
```

1 DiscreteDynamicalSystem

1.1 1D case

[3]:
```
ds = dds(model="logistic map")
```

[4]:
```
x = 0.2
r = 3.8
total_time = int(1e6)
transient_time = int(5e5)
```

[5]:
```
ds.lyapunov(x, 10, parameters=r)
```

[5]:
```
0.39687455416259976
```

[6]:
```
%%timeit -n 5 -r 10
ds.trajectory(x, total_time, parameters=r, transient_time=transient_time)
```

```
21 ms ± 6.33 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[7]:
```
%%timeit -n 5 -r 10
ds.lyapunov(x, total_time, parameters=r, transient_time=transient_time)
```

```
94.2 ms ± 3.94 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[9]:
```
%%timeit -n 5 -r 10
ds.recurrence_time_entropy(x, transient_time + 10000, parameters=r,␣
↪transient_time=transient_time)
```

```
142 ms ± 7.12 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

1.2 2D case

1.2.1 Area-preserving case

[41]:
```
ds = dds(model="standard map")
```

[42]:
```
u = [0.05, 0.05]
k = 1.5
total_time = int(1e6)
```

[ ]:
```
ds.lyapunov(u, 10, parameters=k)
ds.dig(u, 10, parameters=k)
ds.hurst_exponent(u, 10, parameters=k)
```

[ ]:
```
0.2968127463008324
```

[44]:
```
%%timeit -n 5 -r 10
ds.trajectory(u, total_time, parameters=k)
```

```
35.9 ms ± 1.14 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[45]:
```
%%timeit -n 5 -r 10
ds.lyapunov(u, total_time, parameters=k)
```

```
345 ms ± 3.2 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[50]:
```
%%timeit -n 5 -r 10
ds.SALI(u, total_time, parameters=k)
```

```
125  s ± 17.1  s per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[51]:
```
%%timeit -n 5 -r 10
ds.LDI(u, total_time, 2, parameters=k)
```

```
1.47 ms ± 591  s per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[52]:
```
%%timeit -n 5 -r 10
ds.dig(u, total_time, parameters=k)
```

```
47.1 ms ± 1.23 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[53]:
```
%%timeit -n 5 -r 10
ds.recurrence_time_entropy(u, 10000, parameters=k)
```

```
78.8 ms ± 330  s per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[ ]:
```
%%timeit -n 5 -r 10
ds.hurst_exponent(u, total_time, parameters=k)
```

1.2.2 Dissipative case

[29]:
```
ds = dds(model="henon map")
```

[30]:
```
u = [0.05, 0.05]
alpha = 1.4
beta = 0.3
parameters = [alpha, beta]
total_time = int(1e6)
transient_time = int(5e5)
```

[36]:
```
%%timeit -n 5 -r 10
ds.trajectory(u, total_time, parameters=parameters,␣
 ↪transient_time=transient_time)
```

```
44.5 ms ± 923  s per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[37]:
```
%%timeit -n 5 -r 10
ds.lyapunov(u, total_time, parameters=parameters, transient_time=transient_time)
```

```
197 ms ± 2.21 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[38]:
```
%%timeit -n 5 -r 10
ds.SALI(u, total_time, parameters=parameters, transient_time=transient_time)
```

```
20.8 ms ± 87.4  s per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[39]:
```
%%timeit -n 5 -r 10
ds.LDI(u, total_time, 2, parameters=parameters, transient_time=transient_time)
```

```
/opt/anaconda3/lib/python3.12/site-packages/numba/core/utils.py:661:
NumbaExperimentalFeatureWarning: First-class function type feature is

experimental
  warnings.warn("First-class function type feature is experimental",

209 ms ± 7.2 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[40]:
```
%%timeit -n 5 -r 10
ds.recurrence_time_entropy(u, transient_time + 10000, parameters=parameters,␣
 ↪transient_time=transient_time)
```

```
133 ms ± 9.93 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

1.3 3D case

[2]:
```
@njit
def henon_map_3D(u, parameters):
    x, y, z = u
    M1, M2, B = parameters
```

```
    x_new = y
    y_new = z
    z_new = M1 + B * x + M2 * y - z**2

    return np.array([x_new, y_new, z_new], dtype=np.float64)


@njit
def henon_map_3D_jacobian(u, parameters, *args):
    M1, M2, B = parameters

    J = np.array(
        [[0.0, 1.0, 0.0], [0.0, 0.0, 1.0], [B, M2, -2.0 * u[2]]], dtype=np.
 ↪float64
    )

    return J
```

[5]:
```
ds = dds(mapping=henon_map_3D, jacobian=henon_map_3D_jacobian,␣
 ↪system_dimension=3, number_of_parameters=3)
```

[6]:
```
parameters = [0, 0.85, 0.7]
u = [0.6, 0.2, 0.3]
total_time = int(1e6)
transient_time = int(5e5)
```

[13]:
```
%%timeit -n 5 -r 10
ds.trajectory(u, total_time, parameters=parameters)
```

```
44.5 ms ± 786  s per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[14]:
```
%%timeit -n 5 -r 10
ds.lyapunov(u, total_time, parameters=parameters, transient_time=transient_time)
```

```
1.28 s ± 12.6 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[15]:
```
%%timeit -n 5 -r 10
ds.SALI(u, total_time, parameters=parameters, transient_time=transient_time)
```

```
20.1 ms ± 185  s per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[16]:
```
%%timeit -n 5 -r 10
ds.LDI(u, total_time, 2, parameters=parameters, transient_time=transient_time)
```

```
225 ms ± 4.03 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[17]:
```
%%timeit -n 5 -r 10
ds.LDI(u, total_time, 3, parameters=parameters, transient_time=transient_time)
```

```
200 ms ± 7.88 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[19]:
```
%%timeit -n 5 -r 10
ds.recurrence_time_entropy(u, transient_time + 10000, parameters=parameters,␣
↪transient_time=transient_time)
```

```
110 ms ± 5.11 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

1.4 4D case

[28]:
```
ds = dds(model="4d symplectic map")
```

[29]:
```
u = [3.0, 0.0, 0.5, 0.0]
parameters = [0.5, 0.1, 0.001]
total_time = int(1e6)
```

[16]:
```
%%timeit -n 5 -r 10
ds.trajectory(u, total_time, parameters=parameters)
```

```
56.6 ms ± 605  s per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[17]:
```
%%timeit -n 5 -r 10
ds.lyapunov(u, total_time, parameters=parameters)
```

```
3.77 s ± 168 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[18]:
```
%%timeit -n 5 -r 10
ds.SALI(u, total_time, parameters=parameters)
```

```
4.65 ms ± 143  s per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[32]:
```
%%timeit -n 5 -r 10
ds.LDI(u, total_time, 2, parameters=parameters)
```

```
129 ms ± 1.3 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[20]:
```
%%timeit -n 5 -r 10
ds.LDI(u, total_time, 3, parameters=parameters)
```

```
59.9 ms ± 805  s per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[31]:
```
%%timeit -n 5 -r 10
ds.LDI(u, total_time, 4, parameters=parameters)
```

```
13.6 ms ± 1.08 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[21]:
```
%%timeit -n 5 -r 10
ds.recurrence_time_entropy(u, 10000, parameters=parameters)
```

```
98.2 ms ± 554  s per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[33]:
```
10000 + 5e5
```

[33]:
```
510000.0
```

2 ContinuousDynamicalSystem

2.1 Lorenz system

[6]:
```
ds = cds(model="lorenz system")
ds.integrator("rk4", time_step=0.01)
```

[7]:
```
u = [0.0, 0.1, 0.0]
parameters = [10, 28, 8/3]
total_time = 1e4
transient_time = 5e3
```

[9]:
```
%%timeit -n 5 -r 10
ds.trajectory(u, total_time, parameters=parameters,␣
 ↪transient_time=transient_time)
```

```
574 ms ± 3.36 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[10]:
```
%%timeit -n 5 -r 10
ds.lyapunov(u, total_time, parameters=parameters, transient_time=transient_time)
```

```
2.18 s ± 63 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[11]:
```
%%timeit -n 5 -r 10
ds.SALI(u, total_time, parameters=parameters, transient_time=transient_time)
```

```
206 ms ± 61.6 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[12]:
```
%%timeit -n 5 -r 10
ds.LDI(u, total_time, 2, parameters=parameters, transient_time=transient_time)
```

```
344 ms ± 23.8 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[13]:
```
%%timeit -n 5 -r 10
ds.LDI(u, total_time, 3, parameters=parameters, transient_time=transient_time)
```

```
189 ms ± 2.68 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

2.2 Rössler system

[34]:
```
ds = cds(model="rossler system")
ds.integrator("rk4", time_step=0.01)
```

[35]:
```
u = [0.1, 0.1, 0.1]
parameters = [0.15, 0.20, 10.0]
total_time = 1e4
transient_time = 5e3
```

[36]:
```
%%timeit -n 5 -r 10
```

```
ds.trajectory(u, total_time, parameters=parameters,␣
 ↪transient_time=transient_time)
```

```
596 ms ± 50.6 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[37]:
```
%%timeit -n 5 -r 10
ds.lyapunov(u, total_time, parameters=parameters, transient_time=transient_time)
```

```
2.17 s ± 74.9 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[38]:
```
%%timeit -n 5 -r 10
ds.SALI(u, total_time, parameters=parameters, transient_time=transient_time)
```

```
314 ms ± 56 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[39]:
```
%%timeit -n 5 -r 10
ds.LDI(u, total_time, 2, parameters=parameters, transient_time=transient_time)
```

```
1.92 s ± 31.1 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[40]:
```
%%timeit -n 5 -r 10
ds.LDI(u, total_time, 3, parameters=parameters, transient_time=transient_time)
```

```
201 ms ± 2.12 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

2.3 4D Rössler system

[41]:
```
ds = cds(model="4d rossler system")
ds.integrator("rk4", time_step=0.01)
```

[50]:
```
u = [-20., 0, 0., 15.]
parameters = [0.25, 3.0, 0.5, 0.05]
total_time = 1e4
transient_time = 5e3
```

[51]:
```
%%timeit -n 5 -r 10
ds.trajectory(u, total_time, parameters=parameters,␣
 ↪transient_time=transient_time)
```

```
595 ms ± 5.95 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[52]:
```
%%timeit -n 5 -r 10
ds.lyapunov(u, total_time, parameters=parameters, transient_time=transient_time)
```

```
2.82 s ± 54.8 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[53]:
```
%%timeit -n 5 -r 10
ds.SALI(u, total_time, parameters=parameters, transient_time=transient_time)
```

```
296 ms ± 2.88 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[54]:
```
%%timeit -n 5 -r 10
ds.LDI(u, total_time, 2, parameters=parameters, transient_time=transient_time)
```

```
2.07 s ± 31.5 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[55]:
```
%%timeit -n 5 -r 10
ds.LDI(u, total_time, 3, parameters=parameters, transient_time=transient_time)
```

```
1.21 s ± 23.6 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[56]:
```
%%timeit -n 5 -r 10
ds.LDI(u, total_time, 4, parameters=parameters, transient_time=transient_time)
```

```
194 ms ± 8.81 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

2.4 Hénon-Heiles

[3]:
```
ds = cds(model="henon heiles")
ds.integrator("rk4", time_step=0.01)
```

[4]:
```
E_ref = 1 / 8  # Total energy of the system
x = 0  # Define the initial condition
y = -0.25
py = 0
px = np.sqrt(2 * (E_ref - x**2 * y + y**3 / 3) - x**2 - y**2 - py**2)

dof = 2
q = np.array([x, y])
p = np.array([px, py])

u = np.array([x, y, px, py])

total_time = 10000
num_intersections = 5000
```

[6]:
```
%%timeit -n 5 -r 10
ds.trajectory(u, total_time)
```

```
837 ms ± 12.3 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[8]:
```
%%timeit -n 5 -r 10
ds.lyapunov(u, total_time)
```

```
16 s ± 128 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[10]:
```
%%timeit -n 5 -r 10
ds.SALI(u, total_time)
```

```
211 ms ± 2.31 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[12]:
```
%%timeit -n 5 -r 10
ds.LDI(u, total_time, 2)
```

```
1.83 s ± 23.4 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[13]:
```
%%timeit -n 5 -r 10
ds.LDI(u, total_time, 3)
```

```
799 ms ± 10.9 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[14]:
```
%%timeit -n 5 -r 10
ds.LDI(u, total_time, 4)
```

```
330 ms ± 3.44 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[7]:
```
%%timeit -n 5 -r 10
ds.poincare_section(u, num_intersections, 0, 0)
```

```
933 ms ± 17.2 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

3 HamiltonianSystem

[3]:
```
E_ref = 1 / 8  # Total energy of the system
x = 0  # Define the initial condition
y = -0.25
py = 0
px = np.sqrt(2 * (E_ref - x**2 * y + y**3 / 3) - x**2 - y**2 - py**2)

dof = 2
q = np.array([x, y])
p = np.array([px, py])

u = np.array([x, y, px, py])

num_intersections = 5000
total_time = 10000
time_step = 0.01
```

[4]:
```
hs = HS(model="henon heiles")
```

3.1 Yoshida 4th order

[5]:
```
hs.integrator("svy4", time_step=time_step)
```

[6]:
```
%%timeit -n 5 -r 10
hs.trajectory(q, p, total_time)
```

```
699 ms ± 71.6 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[7]:
```
%%timeit -n 5 -r 10
hs.poincare_section(q, p, num_intersections)
```

```
2.16 s ± 19 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[8]:
```
%%timeit -n 5 -r 10
hs.lyapunov(q, p, total_time)
```

```
13.5 s ± 104 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[9]:
```
%%timeit -n 5 -r 10
hs.SALI(q, p, total_time)
```

```
576 ms ± 24.9 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[10]:
```
%%timeit -n 5 -r 10
hs.LDI(q, p, total_time, 2)
```

```
2.57 s ± 13.9 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[11]:
```
%%timeit -n 5 -r 10
hs.LDI(q, p, total_time, 3)
```

```
1.57 s ± 4.78 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[12]:
```
%%timeit -n 5 -r 10
hs.LDI(q, p, total_time, 4)
```

```
805 ms ± 18.1 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

3.2 Velocity-Verlet 2nd order

[13]:
```
hs.integrator("vv2", time_step=time_step)
```

[14]:
```
%%timeit -n 5 -r 10
hs.trajectory(q, p, total_time)
```

```
216 ms ± 9.23 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[15]:
```
%%timeit -n 5 -r 10
hs.poincare_section(q, p, num_intersections)
```

```
682 ms ± 7.76 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[16]:
```
%%timeit -n 5 -r 10
hs.lyapunov(q, p, total_time)
```

```
6.18 s ± 31.7 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[17]:
```
%%timeit -n 5 -r 10
hs.SALI(q, p, total_time)
```

```
318 ms ± 20.5 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[18]:
```
%%timeit -n 5 -r 10
hs.LDI(q, p, total_time, 2)
```

```
3.65 s ± 14.4 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[19]:
```
%%timeit -n 5 -r 10
hs.LDI(q, p, total_time, 3)
```

```
1.35 s ± 3.99 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```

[20]:
```
%%timeit -n 5 -r 10
hs.LDI(q, p, total_time, 4)
```

```
668 ms ± 2.61 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)
```